TwinOps

What if AI had to prove its fix before touching production?

TwinOps reproduces an incident in an isolated digital twin, tests competing fixes there, attacks its own best answer and hands a human only the change that survived the evidence.

`Design principle: don’t trust the AI’s reasoning. Make it prove the change.`

The problem: fixes that sound right

Under incident pressure, the first fix that restores service wins, whether or not it is safe.

- **Tempting fixes** such as disabling a filter, pinning hosts or turning off TLS validation make the symptom go away and open new risk.
- **Change review is re-investigation.** Approvers redo the diagnosis because there is no shared evidence.
- **AI suggestions are not proof.** A fluent explanation is not evidence, and a model can be steered by text in logs.

REAL EXAMPLE FROM OUR LAB

Disabling the edge filter restores every customer journey *and* exposes the management console to an untrusted edge.

```
NEG-1 admin console 10.99.0.10:8443 answered HTTP from untrusted 10.23.0.1
```

The TwinOps loop

1 Observe

2 Diagnose

3 Reproduce in twin

4 Experiment

5 Challenge

6 Proof Gate

7 Human approval

8 Apply

ALLOWED TO BE WRONG SAFELY

Candidates are tested on an isolated twin. Probes are simulated in the model twin and real network traffic in Containerlab. Failures remain outside the target network.

ATTACKS ITS OWN ANSWER

The challenge role tries IPv6 parity, a secondary node failure, warm caches and session continuity against the best fix.

PRODUCTION STAYS READ-ONLY

The only write path is an apply gateway that needs a human approval bound to the change hash and the target-state hash.

Live demo: one real run on the emulated lab

- Fault reproduced in the twin: 10/10 symptoms match
- A · Restart routing → REJECTED: J1, J3 still fail (TCP timeout, curl exit 28)
- B · Roll back prefix-list → IPv6 challenge fails → superseded
- B′ · Remove EDGE-IN seq 30 only → 7/7 automated gates pass
- U · Disable the filter (evaluation case) → REJECTED: admin console exposed
- Human approves the exact hash → applied → 11 real post-checks pass

58.5 s
historical runbook smoke, 9 Oct 2026 · FRR, CoreDNS, nginx in Containerlab

0
production writes during diagnosis

2
unsafe or insufficient fixes rejected with evidence

Architecture

Web workbench · React

SSE event stream, evidence tabs, approval dialog



FastAPI + SSE

Cloud Run packaging; deployment unverified · incident evidence in local SQLite

Planner

Ollama tested locally · Gemini/Vertex adapters · bounded feedback and runbook fallback

Twin backend

model twin (Cloud Run) · Containerlab lab: FRR BGP, CoreDNS, nginx TLS

Proof Gate G1–G8

deterministic code: reproduction, restore, regression, challenges, policy, rollback, latency, human

Apply gateway

approval bound to change + state · post-checks · verified rollback or operator recovery

Same event and evidence model in the browser engine, the Python engine and the lab, checked by 40 parity cases.

AI proposes. Evidence decides.

BOUNDED MODEL REASONING

- Ranked hypotheses that cite evidence IDs (JSON schema)
- Fix proposals from a typed action catalogue
- Failed tests guide a bounded repair and fresh retest
- Approver briefing, labelled non-authoritative

CODE DOES THE JUDGING

- Twin, probes, gates and policy are deterministic
- Catalogue actions and source IDs checked; hypotheses remain interpretations
- Device logs are fenced as untrusted data
- Historical fixes retested; past approval never reused

Evidence: evaluation harness

Historical runbook lab smoke · 9 Oct 2026 · one run per fault

FAULT	VERDICT	COMPUTE
F1 · Route misconfiguration B superseded by B′ after a real IPv6 timeout	PASS UNDER DEFINED TESTS	58.5 s
F2 · DNS failure C superseded by C′: warm resolver cache still stale	PASS UNDER DEFINED TESTS	41.1 s
F3 · Certificate expiry reload drains long-lived sessions (G7 warning)	PASS WITH WARNINGS	34.4 s
F0 · Link flap (unsupported) not reproducible in the twin, so no fix proposed	INCONCLUSIVE	18.3 s

OLLAMA · QWEN2.5:7B VS RUNBOOK

core · standard · seeds 1001–1001

FAULT	RUNBOOK	MODEL, NO FALLBACK	UNSAFE PASS
route misconfig	1/1	0/1	0
dns failure	1/1	0/1	0
cert expiry	1/1	1/1	0
unsupported link flap	1/1	1/1	0

llm-planner-0.3.0 · evaluation-0.2.0 · measured 10/10/2026. Smoke sample only, not a reliability estimate. Memory disabled in paired evaluations.

- Target drifts after the snapshot → approval invalidated
- Approval hash ≠ tested change → refused
- Operator rejects → zero writes
- Planner emits an off-catalogue action → never executed

Impact on enterprise operations work

ON-CALL ENGINEER

Starts from a reproduced fault and tested candidates instead of a blank terminal. The tempting fix is rejected before anyone tries it.

CHANGE APPROVER

Reviews a Proof Pack covering evidence, diff, rollback rehearsal and untested conditions, then approves a hash rather than a story.

ORGANISATION

Past successful changes become candidates for fresh verification. A similar incident never imports an old approval.

A read-only review trial records time and gate agreement for the same candidate with raw evidence and with its Proof Pack. Human trials are pending; no time savings are claimed.

Scalability: packs and twin backends

- **Scenario packs**: regional telecom core and hybrid enterprise today. The topology, journeys and fault catalogue are data.
- **Twin backends** behind one protocol: an in-process model twin, then a Containerlab lab with real routing, DNS and TLS, then Kubernetes (kind), cloud-config sandboxes and CI/CD.
- **Planner providers**: Gemini API, Vertex AI, any OpenAI-compatible endpoint, or replay. The planner can change without touching the Proof Gate.

HONEST LIMITS TODAY

- Local SQLite preserves evidence. Cloud Run instance replacement needs external storage; one service process owns a target.
- Surge and long-lived session behaviour are listed as untested in every Proof Pack.
- The lab is synthetic. Production would start in read-only shadow mode.

Roadmap and ask

1. Now built: incident-memory retesting, changed-state challenges and anonymous review trials
2. Policy as code (OPA) and real telemetry (Prometheus, OpenTelemetry)
3. Read-only shadow mode next to a real NOC: verdicts recorded, nothing applied

Ask: one operations team, an approved LLM endpoint and a lab slice for a four-week shadow pilot.

> “The AI isn’t impressive because it knew the answer. It’s impressive because it was allowed to be wrong safely until it could prove one.”